Relative path: reasoning/llm_layer.py
Filename: llm_layer.py Extension: .py

```
"""
OASIS v2 — Layer 4: Reasoning & Recommendation
================================================
Takes a RiskAssessment (Layer 3's output) and produces an IncidentNarrative,
grounded ONLY in the RiskAssessment JSON — nothing else.

Two modes:
  - LLM mode: used automatically if ANTHROPIC_API_KEY is set and the
    `anthropic` package is installed.
  - Template mode: fully offline deterministic fallback, always available.

Groundedness check: verifies no entity_id or MITRE technique ID appears in
the output that isn't the one actually in the input JSON.
"""
from __future__ import annotations

import json
import os
import re
from typing import Optional

ENTITY_ID_PATTERN = re.compile(r"\b(?:user|svc|edge)_\d+\b")
MITRE_ID_PATTERN = re.compile(r"\bT\d{3,4}(?:\.\d+)?\b")

PROMPT_TEMPLATE = """You are a SOC analyst assistant. You will be given a single JSON object
describing one flagged session (a "risk assessment"). Write a short incident
narrative based STRICTLY on the fields in this JSON. Do not invent any
entity names, asset names, IPs, or facts that are not present in the JSON.
Do not reference any other session or entity.

RiskAssessment JSON:
{risk_json}

Respond with ONLY a JSON object (no markdown fences, no preamble) with these
exact fields:
{{
  "summary": "<1-2 sentence plain-English summary of what happened>",
  "why_flagged": "<1-2 sentences citing the specific contributing_features and anomaly_type from the JSON>
",
  "recommended_action": "<one of: acknowledge, isolate, escalate -- must match the JSON's recommended_acti
on field>",
  "confidence": <float 0-1, base this on type_confidence and impact_score from the JSON>
}}
"""


def _template_narrative(risk: dict) -> dict:
    """Fully offline fallback — always grounded by construction, since it
    only ever inserts fields straight from `risk`."""
    feats = ", ".join(risk["contributing_features"][:3]) or "no specific features recorded"
    mitre = f"{risk['mitre_technique_name']} ({risk['mitre_technique_id']})" \
        if risk.get("mitre_technique_id") else "no MITRE mapping"
    summary = (
        f"Session {risk['session_id']} for entity {risk['entity_id']} was flagged as "
        f"'{risk['anomaly_type']}' with anomaly_score {risk['anomaly_score']} and "
        f"impact_score {risk['impact_score']}."
    )
    why = (
        f"Classified as {risk['anomaly_type']} (confidence {risk['type_confidence']}), "
        f"mapped to {mitre}. Top contributing features: {feats}. "
        f"Affected asset criticality: {risk['asset_criticality']}/5."
    )
    return {
        "session_id": risk["session_id"],
        "summary": summary,
        "why_flagged": why,
        "recommended_action": risk["recommended_action"],
        "confidence": round(float(risk["type_confidence"]) * (0.5 + 0.5 * risk["impact_score"]), 4),
    }


def _llm_narrative(risk: dict) -> Optional[dict]:
    api_key = os.environ.get("ANTHROPIC_API_KEY")
```

```
    if not api_key:
        return None
    try:
        import anthropic  # pip install anthropic
    except ImportError:
        print("  [reasoning] `anthropic` not installed -- falling back to template mode.")
        return None
    try:
        client = anthropic.Anthropic(api_key=api_key)
        prompt = PROMPT_TEMPLATE.format(risk_json=json.dumps(risk, indent=2))
        response = client.messages.create(
            model="claude-sonnet-4-6",
            max_tokens=400,
            messages=[{"role": "user", "content": prompt}],
        )
        text = "".join(b.text for b in response.content if b.type == "text").strip()
        text = text.strip("`")
        if text.startswith("json"):
            text = text[4:].strip()
        parsed = json.loads(text)
        parsed["session_id"] = risk["session_id"]
        return parsed
    except Exception as e:
        print(f"  [reasoning] LLM call failed ({e}) -- falling back to template mode.")
        return None


def groundedness_check(narrative: dict, risk: dict) -> dict:
    text = " ".join([narrative.get("summary", ""), narrative.get("why_flagged", "")])
    violations = []

    found_entity_ids = set(m.group(0) for m in ENTITY_ID_PATTERN.finditer(text))
    for eid in found_entity_ids:
        if eid != risk["entity_id"]:
            violations.append(f"mentions entity '{eid}' not present in input (expected '{risk['entity_id']
}')")

    found_mitre_ids = set(m.group(0) for m in MITRE_ID_PATTERN.finditer(text))
    expected_mitre = risk.get("mitre_technique_id")
    for mid in found_mitre_ids:
        if mid != expected_mitre:
            violations.append(f"mentions MITRE ID '{mid}' not present in input (expected '{expected_mitre}
')")

    return {"passed": len(violations) == 0, "violations": violations}


def generate_incident_narrative(risk: dict) -> dict:
    narrative = _llm_narrative(risk) or _template_narrative(risk)
    check = groundedness_check(narrative, risk)
    narrative["_groundedness_check"] = check
    if not check["passed"]:
        print(f"  [reasoning] WARNING: groundedness check failed for {risk['session_id']}: {check['violati
ons']}")
    return narrative
```